

-- BDR - Atividade Aula 16 --

--Ronaldo de Avila Ribeiro Junior—

-- EXERCÍCIO 1 ---

```
CREATE OR REPLACE FUNCTION bloquear_exclusao()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF OLD.quantidade > 0 THEN
        RAISE EXCEPTION
            'Não é possível excluir o livro "%". Ainda existem % exemplar(es)
            disponível(is).',
            OLD.titulo, OLD.quantidade;
    END IF;
    RETURN OLD;
END;
$;
```

```
DROP TRIGGER IF EXISTS trg_bloquear_exclusao ON livro;
```

```
CREATE TRIGGER trg_bloquear_exclusao
    BEFORE DELETE ON livro
    FOR EACH ROW
```

```
EXECUTE FUNCTION bloquear_exclusao();
```

-- Teste 1 – livro COM exemplares (O Senhor dos Anéis, quantidade=5):
deve BLOQUEAR

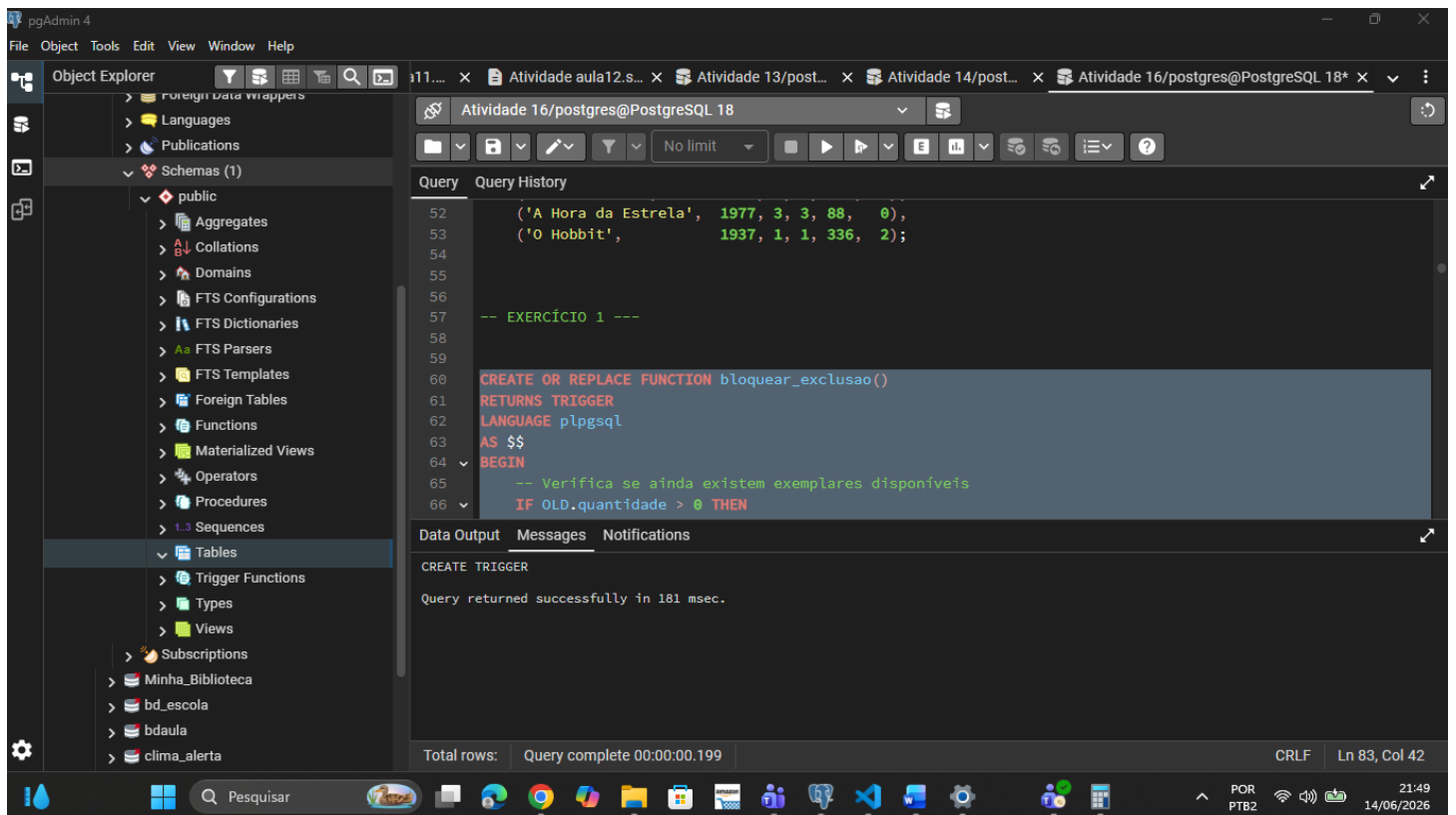
```
DELETE FROM livro WHERE id_livro = 1;
```

-- Teste 2 – livro SEM exemplares (A Hora da Estrela, quantidade=0):
deve PERMITIR

```
DELETE FROM livro WHERE id_livro = 3;
```

-- Verificar resultado

```
SELECT * FROM livro;
```



pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- Foreign data wrappers
- Languages
- Publications
- Schemas (1)
 - public
 - Aggregates
 - Collations
 - Domains
 - FTS Configurations
 - FTS Dictionaries
 - FTS Parsers
 - FTS Templates
 - Foreign Tables
 - Functions
 - Materialized Views
 - Operators
 - Procedures
 - Sequences
 - Tables
 - Trigger Functions
 - Types
 - Views
 - Minha_Biblioteca
 - bd_escola
 - bdaula
 - clima_alerta

Atividade 16/postgres@PostgreSQL 18

Query

```
FOR EACH ROW
EXECUTE FUNCTION bloquear_exclusao();

-- Teste 1 - livro COM exemplares (O Senhor dos Anéis, quantidade=5): deve BLOQUEAR
DELETE FROM livro WHERE id_livro = 1;

-- Teste 2 - livro SEM exemplares (A Hora da Estrela, quantidade=0): deve PERMITIR
DELETE FROM livro WHERE id_livro = 3;

-- Verificar resultado
SELECT * FROM livro;
```

Data Output

id_livro [PK] integer	titulo character varying (150)	ano_publicacao integer	id_autor integer	id_editora integer	num_paginas integer	quantidade integer
1	O Senhor dos Anéis	1954	1	1	1568	5
2	Dom Casmurro	1899	2	2	208	3
3	A Hora da Estrela	1977	3	3	88	0
4	O Hobbit	1937	1	1	336	2

Total rows: 4 Query complete 00:00:00.212 CRLF Ln 92, Col 21

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- Foreign data wrappers
- Languages
- Publications
- Schemas (1)
 - public
 - Aggregates
 - Collations
 - Domains
 - FTS Configurations
 - FTS Dictionaries
 - FTS Parsers
 - FTS Templates
 - Foreign Tables
 - Functions
 - Materialized Views
 - Operators
 - Procedures
 - Sequences
 - Tables
 - Trigger Functions
 - Types
 - Views
 - Minha_Biblioteca
 - bd_escola
 - bdaula
 - clima_alerta

Atividade 16/postgres@PostgreSQL 18

Query

```
FOR EACH ROW
EXECUTE FUNCTION bloquear_exclusao();

-- Teste 1 - livro COM exemplares (O Senhor dos Anéis, quantidade=5): deve BLOQUEAR
DELETE FROM livro WHERE id_livro = 1;

-- Teste 2 - livro SEM exemplares (A Hora da Estrela, quantidade=0): deve PERMITIR
DELETE FROM livro WHERE id_livro = 3;

-- Verificar resultado
SELECT * FROM livro;
```

Data Output

ERROR: Não é possível excluir o livro "O Senhor dos Anéis". Ainda existem 5 exemplar(es) disponível(is).
CONTEXT: PL/pgSQL function bloquear_exclusao() line 5 at RAISE
SQL state: P0001

Total rows: Query complete 00:00:00.069 CRLF Ln 86, Col 40

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- Foreign Data Wrappers
- Languages
- Publications
- Schemas (1)
 - public
 - Aggregates
 - Collations
 - Domains
 - FTS Configurations
 - FTS Dictionaries
 - FTS Parsers
 - FTS Templates
 - Foreign Tables
 - Functions
 - Materialized Views
 - Operators
 - Procedures
 - Sequences
 - Tables
 - Trigger Functions
 - Types
 - Views
 - Subscriptions
 - Minha_Biblioteca
 - bd_escola
 - bdaula
 - clima_alerta

Atividade 16/postgres@PostgreSQL 18

Query

```
FOR EACH ROW
EXECUTE FUNCTION bloquear_exclusao();

-- Teste 1 - livro COM exemplares (O Senhor dos Anéis, quantidade=5): deve BLOQUEAR
DELETE FROM livro WHERE id_livro = 1;

-- Teste 2 - livro SEM exemplares (A Hora da Estrela, quantidade=0): deve PERMITIR
DELETE FROM livro WHERE id_livro = 3;

-- Verificar resultado
SELECT * FROM livro;
```

Data Output Messages Notifications

DELETE 1

Query returned successfully in 69 msec.

Total rows: Query complete 00:00:00.069 CRLF Ln 90, Col 1

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- Foreign Data Wrappers
- Languages
- Publications
- Schemas (1)
 - public
 - Aggregates
 - Collations
 - Domains
 - FTS Configurations
 - FTS Dictionaries
 - FTS Parsers
 - FTS Templates
 - Foreign Tables
 - Functions
 - Materialized Views
 - Operators
 - Procedures
 - Sequences
 - Tables
 - Trigger Functions
 - Types
 - Views
 - Subscriptions
 - Minha_Biblioteca
 - bd_escola
 - bdaula
 - clima_alerta

Atividade 16/postgres@PostgreSQL 18

Query

```
FOR EACH ROW
EXECUTE FUNCTION bloquear_exclusao();

-- Teste 1 - livro COM exemplares (O Senhor dos Anéis, quantidade=5): deve BLOQUEAR
DELETE FROM livro WHERE id_livro = 1;

-- Teste 2 - livro SEM exemplares (A Hora da Estrela, quantidade=0): deve PERMITIR
DELETE FROM livro WHERE id_livro = 3;

-- Verificar resultado
SELECT * FROM livro;
```

Data Output Messages Notifications

Showing rows: 1 to 3 of 1 Page No: 1

id_livro [PK] integer	titulo character varying (150)	ano_publicacao integer	id_autor integer	id_editora integer	num_paginas integer	quantidade integer
1	O Senhor dos Anéis	1954	1	1	1568	5
2	Dom Casmurro	1899	2	2	208	3
3	O Hobbit	1937	1	1	336	2

Successfully run. Total query runtime: 191 msec. 3 rows affected.

Total rows: 3 Query complete 00:00:00.191 CRLF Ln 92, Col 21

-- EXERCÍCIO 2 – Registro Automático de Exclusões

```
CREATE OR REPLACE FUNCTION log_exclusao_livro()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO log_exclusao_livro (titulo_livro, data_exclusao,
mensagem)
VALUES (
    OLD.titulo,
    NOW(),
    'Livro removido do sistema automaticamente pelo trigger.'
);

    RETURN OLD;
END;
$$;

DROP TRIGGER IF EXISTS trg_log_exclusao ON livro;

CREATE TRIGGER trg_log_exclusao
AFTER DELETE ON livro
```

FOR EACH ROW

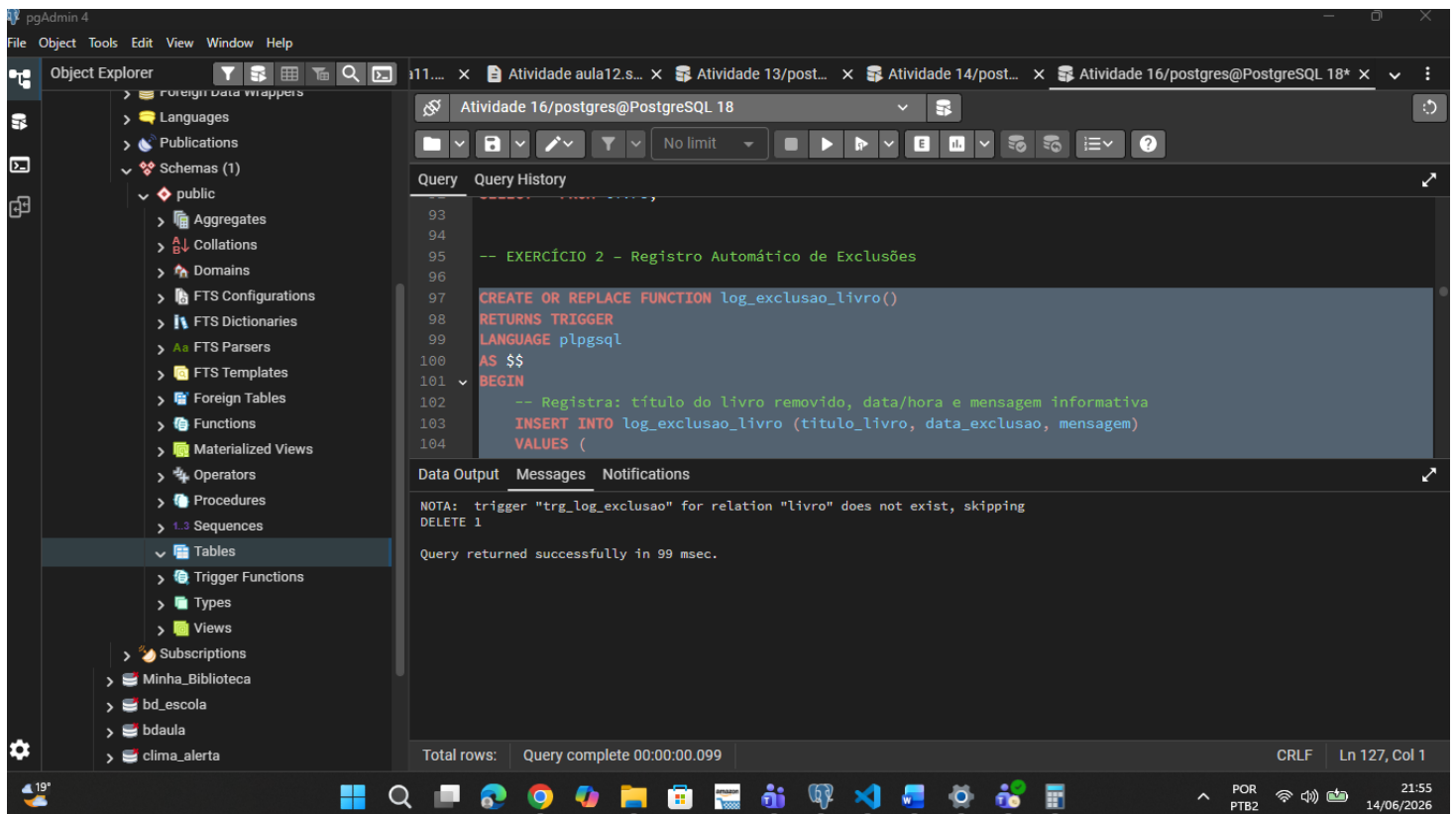
EXECUTE FUNCTION log_exclusao_livro();

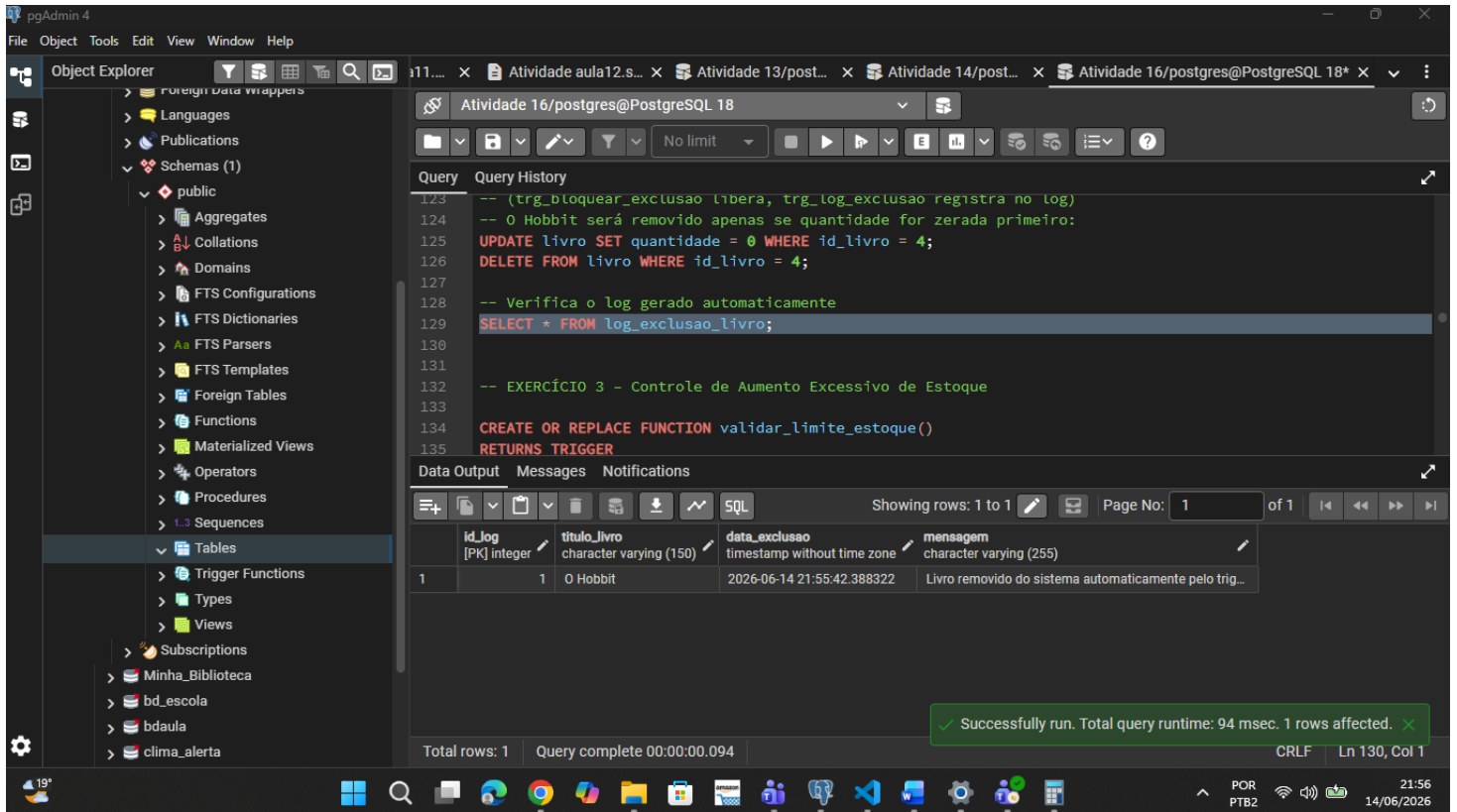
UPDATE livro SET quantidade = 0 WHERE id_livro = 4;

DELETE FROM livro WHERE id_livro = 4;

-- Verifica o log gerado automaticamente

SELECT * FROM log_exclusao_livro;





-- EXERCÍCIO 3 – Controle de Aumento Excessivo de Estoque

CREATE OR REPLACE FUNCTION validar_limite_estoque()

RETURNS TRIGGER

LANGUAGE plpgsql

AS \$\$

BEGIN

IF NEW.quantidade > 100 THEN

RAISE EXCEPTION

'Quantidade inválida! O limite máximo de estoque é 100. Valor informado: %!';

```
        NEW.quantidade;  
    END IF;  
    RETURN NEW;  
END;  
$$;
```

```
DROP TRIGGER IF EXISTS trg_validar_limite ON livro;
```

```
CREATE TRIGGER trg_validar_limite  
    BEFORE UPDATE ON livro  
    FOR EACH ROW  
    EXECUTE FUNCTION validar_limite_estoque();
```

```
-- Teste 1 – valor válido (50): deve PERMITIR o UPDATE
```

```
UPDATE livro SET quantidade = 50 WHERE id_livro = 1;
```

```
-- Teste 2 – valor inválido (200): deve BLOQUEAR com RAISE  
EXCEPTION
```

```
UPDATE livro SET quantidade = 200 WHERE id_livro = 1;
```

```
-- Verificar resultado
```

```
SELECT id_livro, titulo, quantidade FROM livro;
```


pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- Foreign Data Wrappers
- Languages
- Publications
- Schemas (1)
 - public
 - Aggregates
 - Collations
 - Domains
 - FTS Configurations
 - FTS Dictionaries
 - FTS Parsers
 - FTS Templates
 - Foreign Tables
 - Functions
 - Materialized Views
 - Operators
 - Procedures
 - Sequences
 - Tables
 - Trigger Functions
 - Types
 - Views
 - Subscriptions
 - Minha_Biblioteca
 - bd_escola
 - bdaula
 - clima_alerta

Query

```
128 --verifica o log gerado automaticamente
129 SELECT * FROM log_exclusao_livro;
130
131
132 -- EXERCÍCIO 3 - Controle de Aumento Excessivo de Estoque
133
134 CREATE OR REPLACE FUNCTION validar_limite_estoque()
135 RETURNS TRIGGER
136 LANGUAGE plpgsql
137 AS $$
138 BEGIN
```

Query History

Data Output Messages Notifications

NOTA: trigger "trg_validar_limite" for relation "livro" does not exist, skipping
CREATE TRIGGER

Query returned successfully in 95 msec.

Total rows: Query complete 00:00:00.095 CRLF Ln 157, Col 47

19° POR PTB2 21:58 14/06/2026

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- Foreign Data Wrappers
- Languages
- Publications
- Schemas (1)
 - public
 - Aggregates
 - Collations
 - Domains
 - FTS Configurations
 - FTS Dictionaries
 - FTS Parsers
 - FTS Templates
 - Foreign Tables
 - Functions
 - Materialized Views
 - Operators
 - Procedures
 - Sequences
 - Tables
 - Trigger Functions
 - Types
 - Views
 - Subscriptions
 - Minha_Biblioteca
 - bd_escola
 - bdaula
 - clima_alerta

Query

```
159 -- Teste 1 - valor válido (50): deve PERMITIR o UPDATE
160 UPDATE livro SET quantidade = 50 WHERE id_livro = 1;
161
162 -- Teste 2 - valor inválido (200): deve BLOQUEAR com RAISE EXCEPTION
163 UPDATE livro SET quantidade = 200 WHERE id_livro = 1;
164
165 -- Verificar resultado
166 SELECT id_livro, titulo, quantidade FROM livro;
167
168
```

Query History

Data Output Messages Notifications

Showing rows: 1 to 2 Page No: 1 of 1

id_livro	titulo	quantidade
1	O Senhor dos Anéis	5
2	Dom Casmurro	3

Total rows: 2 Query complete 00:00:00.087 CRLF Ln 166, Col 48

19° POR PTB2 21:58 14/06/2026

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- Foreign Data Wrappers
- Languages
- Publications
- Schemas (1)
 - public
 - Aggregates
 - Collations
 - Domains
 - FTS Configurations
 - FTS Dictionaries
 - FTS Parsers
 - FTS Templates
 - Foreign Tables
 - Functions
 - Materialized Views
 - Operators
 - Procedures
 - Sequences
 - Tables
 - Trigger Functions
 - Types
 - Views
 - Subscriptions
 - Minha_Biblioteca
 - bd_escola
 - bd_aula
 - clima_alerta

Atividade 16/postgres@PostgreSQL 18

Query

```
CREATE TRIGGER trg_validar_limite
BEFORE UPDATE ON livro
FOR EACH ROW
EXECUTE FUNCTION validar_limite_estoque();

-- Teste 1 - valor válido (50): deve PERMITIR o UPDATE
UPDATE livro SET quantidade = 50 WHERE id_livro = 1;

-- Teste 2 - valor inválido (200): deve BLOQUEAR com RAISE EXCEPTION
UPDATE livro SET quantidade = 200 WHERE id_livro = 1;
```

Data Output

UPDATE 1

Query returned successfully in 103 msec.

Total rows: Query complete 00:00:00.103 CRLF Ln 160, Col 53

19° POR PTB2 21:59 14/06/2026

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- Foreign Data Wrappers
- Languages
- Publications
- Schemas (1)
 - public
 - Aggregates
 - Collations
 - Domains
 - FTS Configurations
 - FTS Dictionaries
 - FTS Parsers
 - FTS Templates
 - Foreign Tables
 - Functions
 - Materialized Views
 - Operators
 - Procedures
 - Sequences
 - Tables
 - Trigger Functions
 - Types
 - Views
 - Subscriptions
 - Minha_Biblioteca
 - bd_escola
 - bd_aula
 - clima_alerta

Atividade 16/postgres@PostgreSQL 18

Query

```
-- Teste 1 - valor válido (50): deve PERMITIR o UPDATE
UPDATE livro SET quantidade = 50 WHERE id_livro = 1;

-- Teste 2 - valor inválido (200): deve BLOQUEAR com RAISE EXCEPTION
UPDATE livro SET quantidade = 200 WHERE id_livro = 1;

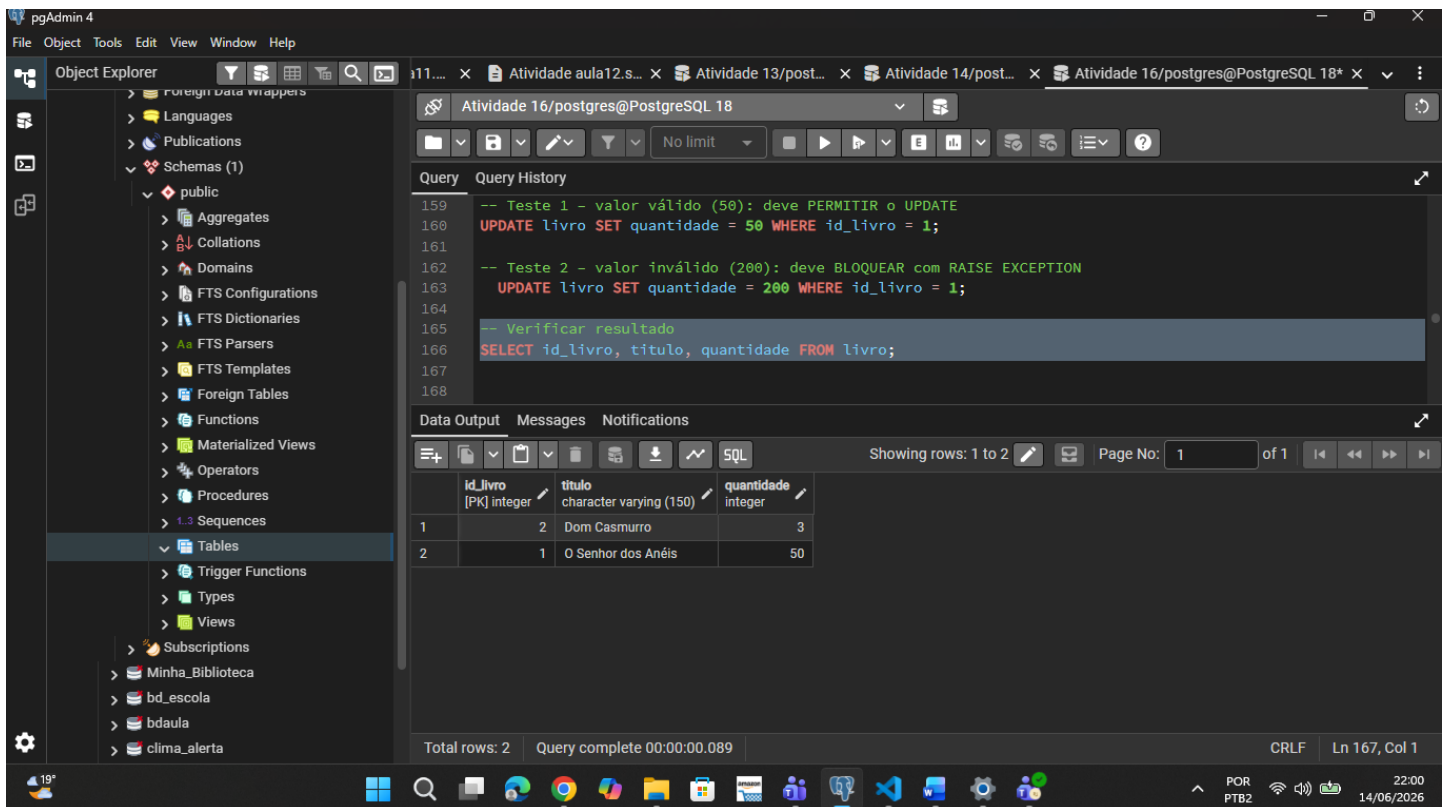
-- Verificar resultado
SELECT id_livro, titulo, quantidade FROM livro;
```

Data Output

ERROR: Quantidade inválida! O limite máximo de estoque é 100. Valor informado: 200.
CONTEXT: PL/pgSQL function validar_limite_estoque() line 5 at RAISE
SQL state: P0001

Total rows: Query complete 00:00:00.066 CRLF Ln 163, Col 56

19° POR PTB2 22:00 14/06/2026



-- EXERCÍCIO 4 – Análise de Trigger BEFORE e AFTER

a) Diferença entre BEFORE e AFTER

BEFORE: Executa antes da alteração ser consolidada.

Permite validar, modificar os dados antes de salvar ou abortar a operação (RAISE EXCEPTION).

AFTER: Executa depois que a alteração já foi concluída. Não pode mais barrar a ação e serve para reagir ao evento.

b) Qual usar para validação?

BEFORE. É a escolha ideal porque impede que dados inválidos cheguem a serem gravados no banco de dados, protegendo a integridade dos dados logo na entrada.

c) Qual usar para auditoria?

AFTER. É a mais indicada porque garante que o log só será gerado se a operação de fato acontecer com sucesso, evitando falsos registros de ações que foram abortadas.

d) Por que a ordem de execução importa?

Porque as triggers rodam em sequência. Se um gatilho BEFORE cancela a ação, os subsequentes não executam. Além disso, em operações encadeadas (onde uma trigger mexe em outra tabela), a ordem errada gera inconsistência de dados e comportamentos imprevisíveis.

-- EXERCÍCIO 5 ---

a) Riscos dessa decisão (Validação apenas na aplicação)

Bypass de regras: Qualquer acesso direto (pgAdmin, scripts, migrações ou APIs externas) insere dados sem validação.

Inconsistência: Risco de gravar dados inválidos, negativos ou duplicados diretamente no banco.

Vulnerabilidade a bugs: Atualizações erradas no código da aplicação podem desativar ou ignorar as regras de validação.

Duplicação e divergência: Se houver múltiplos apps (Web, Mobile, ERP) acessando o mesmo banco, cada um terá que reimplementar as regras, gerando desalinhamento.

b) Vantagens de manter regras no banco

Centralização: As regras ficam em um único lugar e valem para qualquer ferramenta ou sistema que acesse o banco.

Garantia de integridade: Segurança total dos dados, independente de falhas na aplicação ou de quem está manipulando o banco.

Manutenção simplificada: Alterar a regra no banco atualiza o comportamento de todos os sistemas conectados instantaneamente.

Menos código duplicado: Evita replicar a mesma lógica de validação em diferentes backends.

c) Como triggers ajudam na integridade e consistência

Execução automática: Rodam direto no motor do banco, sem depender do desenvolvedor lembrar de chamá-las.

Bloqueio garantido: Triggers BEFORE barram operações proibidas na raiz antes de alterarem o disco.

Auditoria infalível: Triggers AFTER garantem o registro automático de logs logo após o sucesso da operação.

d) Exemplo real em sistemas corporativos

No fluxo de um E-commerce:

Um pedido é feito (INSERT na tabela pedido).

Uma trigger BEFORE valida se há estoque. Se não houver, barra a venda ali mesmo.

Se houver, a operação avança e uma trigger AFTER dá a baixa automática na tabela produto, garantindo o estoque correto para Web, Mobile ou balcão físico.

--Código completo ---

```
DROP TABLE IF EXISTS log_exclusao_livro CASCADE;
```

```
DROP TABLE IF EXISTS livro CASCADE;
```

```
DROP TABLE IF EXISTS autor CASCADE;
```

```
DROP TABLE IF EXISTS editora CASCADE;
```

```
CREATE TABLE autor (  
    id_autor SERIAL PRIMARY KEY,  
    nome CHARACTER VARYING(100)  
);
```

```
CREATE TABLE editora (  
    id_editora SERIAL PRIMARY KEY,  
    nome CHARACTER VARYING(100)  
);
```

```
CREATE TABLE livro (  
    id_livro SERIAL PRIMARY KEY,  
    titulo CHARACTER VARYING(150),  
    ano_publicacao INTEGER,  
    id_autor INTEGER REFERENCES autor(id_autor),  
    id_editora INTEGER REFERENCES editora(id_editora),
```

```
num_paginas  INTEGER,  
quantidade   INTEGER DEFAULT 0  
);
```

```
CREATE TABLE log_exclusao_livro (  
    id_log     SERIAL PRIMARY KEY,  
    titulo_livro CHARACTER VARYING(150),  
    data_exclusao TIMESTAMP,  
    mensagem   CHARACTER VARYING(255)  
);
```

```
INSERT INTO autor (nome) VALUES  
    ('J. R. R. Tolkien'),  
    ('Machado de Assis'),  
    ('Clarice Lispector'),  
    ('J.K. Rowling');
```

```
INSERT INTO editora (nome) VALUES  
    ('HarperCollins'),  
    ('Companhia das Letras'),  
    ('Rocco');
```

```
INSERT INTO livro (titulo, ano_publicacao, id_autor,
id_editora, num_paginas, quantidade) VALUES
('O Senhor dos Anéis', 1954, 1, 1, 1568, 5),
('Dom Casmurro', 1899, 2, 2, 208, 3),
('A Hora da Estrela', 1977, 3, 3, 88, 0),
('O Hobbit', 1937, 1, 1, 336, 2);
```

-- EXERCÍCIO 1 ---

```
CREATE OR REPLACE FUNCTION bloquear_exclusao()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
```

```
    IF OLD.quantidade > 0 THEN
        RAISE EXCEPTION
            'Não é possível excluir o livro "%". Ainda existem %
            exemplar(es) disponível(is).';
```



```
        OLD.titulo, OLD.quantidade;

END IF;


RETURN OLD;

END;

$$;


DROP TRIGGER IF EXISTS trg_bloquear_exclusao ON livro;


CREATE TRIGGER trg_bloquear_exclusao
    BEFORE DELETE ON livro
    FOR EACH ROW
    EXECUTE FUNCTION bloquear_exclusao();


-- Teste 1 – livro COM exemplares (O Senhor dos Anéis,
quantidade=5): deve BLOQUEAR

DELETE FROM livro WHERE id_livro = 1;


-- Teste 2 – livro SEM exemplares (A Hora da Estrela,
quantidade=0): deve PERMITIR

DELETE FROM livro WHERE id_livro = 3;


-- Verificar resultado
```

```
SELECT * FROM livro;
```

-- EXERCÍCIO 2 – Registro Automático de Exclusões

```
CREATE OR REPLACE FUNCTION log_exclusao_livro()
```

```
RETURNS TRIGGER
```

```
LANGUAGE plpgsql
```

```
AS $$
```

```
BEGIN
```

```
    INSERT INTO log_exclusao_livro (titulo_livro, data_exclusao,  
mensagem)
```

```
    VALUES (
```

```
        OLD.titulo,
```

```
        NOW(),
```

```
        'Livro removido do sistema automaticamente pelo trigger.'
```

```
    );
```

```
    RETURN OLD;
```

```
END;
```

```
$$;
```

```
DROP TRIGGER IF EXISTS trg_log_exclusao ON livro;
```

```
CREATE TRIGGER trg_log_exclusao
```

```
    AFTER DELETE ON livro
```

```
    FOR EACH ROW
```

```
    EXECUTE FUNCTION log_exclusao_livro();
```

```
UPDATE livro SET quantidade = 0 WHERE id_livro = 4;
```

```
DELETE FROM livro WHERE id_livro = 4;
```

```
-- Verifica o log gerado automaticamente
```

```
SELECT * FROM log_exclusao_livro;
```

```
-- EXERCÍCIO 3 – Controle de Aumento Excessivo de Estoque
```

```
CREATE OR REPLACE FUNCTION validar_limite_estoque()
```

```
RETURNS TRIGGER
```

```
LANGUAGE plpgsql
```

```
AS $$
```

```
BEGIN
```

```
    IF NEW.quantidade > 100 THEN
```

```
        RAISE EXCEPTION
```

```
'Quantidade inválida! O limite máximo de estoque é  
100. Valor informado: %!,'
```

```
NEW.quantidade;
```

```
END IF;
```

```
RETURN NEW;
```

```
END;
```

```
$$;
```

```
DROP TRIGGER IF EXISTS trg_validar_limite ON livro;
```

```
CREATE TRIGGER trg_validar_limite
```

```
BEFORE UPDATE ON livro
```

```
FOR EACH ROW
```

```
EXECUTE FUNCTION validar_limite_estoque();
```

```
-- Teste 1 – valor válido (50): deve PERMITIR o UPDATE
```

```
UPDATE livro SET quantidade = 50 WHERE id_livro = 1;
```

```
-- Teste 2 – valor inválido (200): deve BLOQUEAR com RAISE  
EXCEPTION
```

```
UPDATE livro SET quantidade = 200 WHERE id_livro = 1;
```

-- Verificar resultado

```
SELECT id_livro, titulo, quantidade FROM livro;
```